

Installation

This file gives the shortest installation path and a basic smoke test. See README Section 1 for the complete commands, explanations, data download, and troubleshooting information.

1. Check the platform

The complete artifact workflow in README Sections 1–3 requires Linux ARM64:

```
uname -m
```

Expected output:

```
aarch64
```

The optional Intel benchmark in README Section 4 instead requires Linux `x86_64` and uses `cosipy-312-intel.yml`.

2. Clone the source repositories

```
cd "${HOME}"
git clone https://github.com/Daisy0419/GRB-Search-Pipeline.git

cd "${HOME}/GRB-Search-Pipeline"
git clone -b tsmap-artifact \
    https://github.com/McKelvey-Engineering-CSE/cosipy.git
```

3. Create the Python environment

Follow README Section 1.1.2 to install Conda if necessary. Then run:

```
cd "${HOME}/GRB-Search-Pipeline"
conda env create -f cosipy-312-deps.yaml
conda activate cosipy-312

cd cosipy
git switch tsmap-artifact
python -m pip install "poetry-core>=2,<3"
python -m pip install --no-deps -e .
python -m pip install notebook
```

4. Build the C++ programs

Install LEMON 1.3.1, HDF5, and HighFive exactly as described in README Section 1.1.3. Set the documented environment variables, then run:

```
cd "${HOME}/GRB-Search-Pipeline/search-planning"
cmake -S . -B build
cmake --build build -j
```

5. Basic installation test

From the repository root, run:

```
cd "${HOME}/GRB-Search-Pipeline"
conda activate cosipy-312

python -c \
    "import cosipy; import hdf5plugin; print('Python environment: OK')"

test -x search-planning/build/sp_train \
    && test -x search-planning/build/sp_verify \
    && echo "C++ executables: OK"
```

Expected output:

```
Python environment: OK
C++ executables: OK
```

6. Obtain the experiment data

Follow README Section 1.2 to download `transients.tar.gz` from the cited Zenodo record, verify its MD5 checksum, and extract it as

```
${HOME}/transients/.
```

7. First artifact run

For the shortest evaluation path, follow README Section 2 to open `precomputed_results/visualize_Results.ipynb`. It reads the distributed results and regenerates Figures 4–11 under `results/figures/`.

For a full regeneration, continue with README Section 3. For the optional mapping benchmark, follow README Section 4.